

Automaty

Limit czasu: 2 sekundy

Limit pamięci: 1024 MiB

Dana jest taśma składająca się z N komórek w rzędzie, ponumerowanych od 0 do $N - 1$ od lewej do prawej. Każda komórka ma unikalną wysokość: wysokość komórki i wynosi p_i , a ciąg $p_0, p_1, \dots, p_{N-1}$ jest permutacją liczb $0, 1, \dots, N - 1$.

Dwie komórki o **indeksach** i oraz j nazywamy *bliskimi* wtedy i tylko wtedy, gdy $|i - j| \leq 1$. W szczególności każda komórka jest bliska samej sobie.

Na taśmie stoi robot, początkowo umieszczony w pewnej komórce. Robot przyjmuje polecenia, z których każde jest jednego z dwóch następujących typów:

- **MAX**: spośród wszystkich komórek bliskich obecnej komórce robota, kolejną pozycją robota będzie jedyna komórka o **maksymalnej** wysokości;
- **MIN**: spośród wszystkich komórek bliskich obecnej komórce robota, kolejną pozycją robota będzie jedyna komórka o **minimalnej** wysokości.

Ponieważ komórka jest bliska samej sobie, polecenie może pozostawić robota w tej samej komórce.

Program to skończony ciąg poleceń, gdzie każde polecenie to **MAX** lub **MIN**. Jeśli robot zaczyna w komórce X i wykonuje program S , kończy pracę w pewnej jednoznacznie określonej komórce, oznaczanej jako $\text{result}(S, X)$.

Zostanie zadanych Q zapytań, z których każde podaje pewien zbiór komórek początkowych o rozmiarze K_i ($0 \leq i < Q$). Dokładniej, i -te zapytanie będzie dotyczyć komórek $x_0, x_1, \dots, x_{K_i-1}$; robot zostanie umieszczony w jednej z nich, ale nie wiadomo z góry w której. Dla każdego zapytania musisz ustalić, czy istnieje program, który przemieści robota do tej samej komórki końcowej, niezależnie od tego, która pozycja początkowa ze zbioru x zostanie użyta.

Formalnie, musisz ustalić, czy istnieje program S taki, że:

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

Permutacja p jest **taka sama** dla wszystkich zapytań.

Zauważ, że nie musisz konstruować programu S , a jedynie stwierdzić, czy taki istnieje.

Szczegóły implementacji

Pierwszą funkcją, którą należy zaimplementować, jest:

```
void initialize(std::vector<int> p)
```

- p : permutacja liczb od 0 do $N - 1$.

Drugą funkcją, którą należy zaimplementować, jest:

```
bool exists_program(std::vector<int> x)
```

- x : tablica określająca komórki w ściśle rosnącym porządku, podana dla zapytania.

Funkcja `initialize` jest wywoływana dokładnie raz, przed jakimikolwiek wywołaniami funkcji `exists_program`.

Funkcja `exists_program` jest wywoływana Q razy, po razie dla każdego zapytania. Musi zwrócić `true`, jeśli istnieje program, po którym robot znajdzie się w tej samej komórce niezależnie od tego, z której z podanych komórek wystartował, oraz `false` w przeciwnym wypadku.

Ograniczenia

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ jest permutacją liczb $0, 1, \dots, N - 1$
- $2 \leq K_i \leq N$
- $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$ dla każdego zapytania
- Suma K_i po wszystkich Q zapytaniach nie przekracza 10^6 .

Przykłady

Wejście przykładowego programu oceniającego 1	Wyjście przykładowego programu oceniającego 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Wejście przykładowego programu oceniającego 2	Wyjście przykładowego programu oceniającego 2
7 3 0 4 2 1 3 5 6 3 0 1 3 3 1 3 4 2 3 6	001

Wyjaśnienie przykładu 1

Dla tego przykładu $p = [0, 2, 1]$. Dla drugiego zapytania robot może wystartować w komórce 0, komórce 1 lub komórce 2. Rozważmy program $[MAX]$.

- Gdy robot zaczyna w komórce 0: komórki bliskie 0 to komórki o indeksach 0 i 1. Gdy robot wykonuje polecenie MAX , ponieważ $p_0 < p_1$, przechodzi do komórki 1.
- Gdy robot zaczyna w komórce 1: komórki bliskie 1 to komórki o indeksach 0, 1 i 2. Gdy robot wykonuje polecenie MAX , ponieważ $p_0 < p_1$ oraz $p_1 > p_2$, pozostaje w komórce 1.
- Gdy robot zaczyna w komórce 2: komórki bliskie 2 to komórki o indeksach 1 i 2. Gdy robot wykonuje polecenie MAX , ponieważ $p_1 > p_2$, przechodzi do komórki 1.

Istnieje program, po którym robot kończy pracę w komórce 1 dla wszystkich trzech komórek początkowych, zatem odpowiedzią dla tego zapytania jest `true`. Inne programy również działają, przykładowo $[MIN, MAX]$, $[MIN, MIN, MAX, MIN, MAX, MIN]$ oraz $[MAX, MIN, MAX]$.

Wyjaśnienie przykładu 2

Dla tego przykładu $p = [0, 4, 2, 1, 3, 5, 6]$.

Dla pierwszych dwóch zapytań można wykazać, że nie istnieje program, po którym robot kończy pracę w tej samej komórce z każdej z podanych komórek początkowych, więc odpowiedzią dla obu jest `false`.

Rozważmy ostatnie zapytanie, dla którego robot może wystartować w komórce 3 lub komórce 6, oraz rozważmy program $[MAX, MAX, MAX]$.

- Gdy robot zaczyna w komórce 3: komórki bliskie 3 to komórki o indeksach 2, 3 i 4. Ponieważ $p_3 < p_4$ oraz $p_2 < p_4$, robot przechodzi do komórki 4. Wykonując kolejne dwa polecenia w analogiczny sposób, kończy pracę w komórce 6.
- Gdy robot zaczyna w komórce 6, pozostaje w komórce 6 przez cały czas i tam kończy pracę.

Dlatego odpowiedzią dla tego zapytania jest `true`. Program $[MIN, MAX, MAX, MAX]$ również działa. Jednak inne programy mogą nie działać. Na przykład, jeśli programem jest $[MAX, MAX]$, robot kończy pracę w komórce 5, gdy zaczyna w komórce 3, ale kończy pracę w komórce 6, gdy zaczyna w komórce 6.

Podzadania

Podzadanie	Punkty	N	Q	K_i	Dodatkowe ograniczenia
0	0	–	–	–	Przykłady z zadania.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Jeśli odpowiedź na zapytanie to <code>true</code> , istnieje poprawny program składający się z pojedynczego polecenia.
2	7				Jeśli odpowiedź na zapytanie to <code>true</code> , istnieje poprawny program składający się z co najwyżej pięciu poleceń.
3	9				–
4	17				–
5	7				$x_1 = x_0 + 1$ dla każdego zapytania.
6	8	$\leq 5\ 000$	$\leq 5 \cdot 10^5$	$\leq N$	–
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$		Permutacja jest rosnąca na początku, potem malejąca, a potem znów rosnąca. Formalnie, istnieją dwa indeksy a, b spełniające $0 < a < b < N - 1$ i $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots > p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$ gdy N jest parzyste lub $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$ gdy N jest nieparzyste.
9	23			–	

Przykładowy program oceniający

Format wejścia jest następujący:

- wiersz 1: dwie liczby całkowite – wartości N oraz Q ;
- wiersz 2: N liczb całkowitych $p_0, p_1, \dots, p_{N-1}$, gdzie p_i to wysokość i -tej komórki;
- wiersz $3 + i$: komórki podane dla i -tego zapytania – liczba całkowita K_i , a po niej w tym samym wierszu K_i liczb całkowitych $x_0, x_1, \dots, x_{K_i-1}$, czyli możliwe komórki początkowe robota.

Format wyjścia jest następujący:

- wiersz 1: ciąg binarny o długości Q , którego i -tym znakiem jest 1, jeśli odpowiedzią na i -te zapytanie jest `true`, oraz 0 w przeciwnym wypadku.